Fundamentos de React y entornos de trabajo

Repaso de JavaScript moderno

FUNDAMENTOS DE JAVASCRIPT ES6+

1. Variables y Scope

let, const y var

```
// ❌ Evitar var (function scope)
var nombre = "Juan";

// ✅ Usar let para variables mutables (block scope)
let edad = 25;
edad = 26; // Permite reasignación

// ✅ Usar const para valores inmutables (block scope)
const PI = 3.14159;
const usuario = { nombre: "Ana", edad: 30 };
```

Diferencias Clave

Declaración	Scope	Reasignación	Hoisting
var	Function	✅	✅
let	Block	✅	❌

const	Block	❌	❌

2. Arrow Functions

```
// Función tradicional
function sumar(a, b) {
    return a + b;
}

// Arrow function
const sumar = (a, b) => a + b;

// Arrow function con múltiples líneas
const procesarUsuario = (usuario) => {
    const nombreCompleto = `${usuario.nombre}
${usuario.apellido}`;
    return {
        ...usuario,
        nombreCompleto
    };
};
```

Características importantes:

- No tienen su propio this
- No se pueden usar como constructores
- Ideales para callbacks y funciones cortas

3. Template Literals

```
const nombre = "María";
const edad = 28;
```

```
// ❌ Concatenación tradicional
const mensaje1 = "Hola, soy " + nombre + " y tengo " + edad + "
años";

// ✅ Template literals
const mensaje2 = `Hola, soy ${nombre} y tengo ${edad} años`;

// Múltiples líneas
const html = `
    <div>
        <h1>${nombre}</h1>
        <p>Edad: ${edad}</p>
    </div>
`;
```

4. Destructuring

Destructuring de Objetos

```
const usuario = {
    nombre: "Carlos",
    email: "carlos@email.com",
    edad: 32,
    direccion: {
        ciudad: "Madrid",
        pais: "España"
    }
};

// Destructuring básico
const { nombre, email } = usuario;

// Con renombrado
const { nombre: nombreUsuario, edad: años } = usuario;
```

```
// Destructuring anidado
const { direccion: { ciudad } } = usuario;

// Con valores por defecto
const { telefono = "No especificado" } = usuario;
```

Destructuring de Arrays

```
const colores = ["rojo", "verde", "azul", "amarillo"];

// Destructuring básico
const [primero, segundo] = colores;

// Saltar elementos
const [, , tercero] = colores;

// Rest operator
const [principal, ...secundarios] = colores;
```

5. Spread Operator y Rest Parameters

Spread Operator (...)

```
// Con arrays
const numeros = [1, 2, 3];
const masNumeros = [...numeros, 4, 5];

// Con objetos
const persona = { nombre: "Ana", edad: 25 };
const personaCompleta = {
    ...persona,
    email: "ana@email.com",
    activo: true
```

```
};

// En llamadas a funciones
const max = Math.max(...numeros);
```

Rest Parameters

```
// Función que acepta cualquier cantidad de argumentos
const sumarTodos = (...numeros) => {
    return numeros.reduce((total, num) => total + num, 0);
};

sumarTodos(1, 2, 3, 4, 5); // 15
```

6. Array Methods (Programación Funcional)

Métodos Esenciales

```
const productos = [
    { id: 1, nombre: "Laptop", precio: 1000, categoria: "tech"
},
    { id: 2, nombre: "Mouse", precio: 25, categoria: "tech" },
    { id: 3, nombre: "Libro", precio: 15, categoria: "educacion"
},
    { id: 4, nombre: "Teclado", precio: 75, categoria: "tech" }
];

// map - Transformar elementos
const nombres = productos.map(p => p.nombre);
const productosConDescuento = productos.map(p => ({
    ...p,
    precioConDescuento: p.precio * 0.9
}));
```

```
// filter - Filtrar elementos
const productosTech = productos.filter(p => p.categoria ===
"tech");
const productosCaros = productos.filter(p => p.precio > 50);

// find - Encontrar un elemento
const laptop = productos.find(p => p.nombre === "Laptop");

// reduce - Reducir a un valor
const precioTotal = productos.reduce((total, p) => total +
p.precio, 0);

// some y every
const hayProductosCaros = productos.some(p => p.precio > 500);
const todosBaratos = productos.every(p => p.precio < 2000);
```

7. Promises y Async/Await

Promises

```
// Crear una promise
const obtenerUsuario = (id) => {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            if (id > 0) {
                resolve({ id, nombre: `Usuario ${id}` });
            } else {
                reject(new Error("ID inválido"));
            }
        }, 1000);
    });
};

// Usar promises
obtenerUsuario(1)
```

```
    .then(usuario => console.log(usuario))
    .catch(error => console.error(error));
```

Async/Await

```
// Función async
const obtenerDatosUsuario = async (id) => {
    try {
        const usuario = await obtenerUsuario(id);
        const permisos = await obtenerPermisos(usuario.id);
        return { ...usuario, permisos };
    } catch (error) {
        console.error("Error:", error.message);
        throw error;
    }
};

// Usar async/await
const cargarDatos = async () => {
    const datos = await obtenerDatosUsuario(1);
    console.log(datos);
};
```

8. Modules (Import/Export)

Export

```
// utils.js
export const PI = 3.14159;

export const calcularArea = (radio) => PI * radio * radio;

export default class Calculadora {
    static sumar(a, b) {
        return a + b;
    }
```

```
}
```

Import

```
// app.js
import Calculadora, { PI, calcularArea } from './utils.js';

// Import con alias
import { calcularArea as area } from './utils.js';

// Import todo
import * as Utils from './utils.js';
```

CONCEPTOS ESENCIALES DE REACT

1. Componentes

Componentes Funcionales

```
// Componente básico
const Saludo = () => {
    return <h1>¡Hola Mundo!</h1>;
};

// Componente con props
const Usuario = ({ nombre, edad, activo = true }) => {
    return (
        <div className={`usuario ${activo ? 'activo' :
'inactivo'}`}>
            <h2>{nombre}</h2>
            <p>Edad: {edad}</p>
            <span>{activo ? "En línea" : "Desconectado"}</span>
        </div>
    );
};
```

```
// Uso del componente
const App = () => {
    return (
        <div>
            <Usuario nombre="Ana" edad={25} />
            <Usuario nombre="Carlos" edad={30} activo={false} />
        </div>
    );
};
```

2. NodeJS

Es un entorno de ejecución de JavaScript fuera del navegador. Permite ejecutar código JS en el servidor gracias a su motor V8 de Google Chrome.

Para qué sirve: Crear aplicaciones backend, scripts, herramientas de línea de comandos, y ejecutar código que normalmente solo correría en el navegador.

En React: Se usa para levantar el servidor de desarrollo, compilar el código con herramientas como Webpack/Vite y generar el build final.

3. NPM

Es el gestor de paquetes que viene incluido con Node.js. Permite instalar, actualizar y gestionar librerías y dependencias de un proyecto.

Para qué sirve: Acceder a miles de paquetes reutilizables, manejar sus versiones y scripts de proyecto.

En React: Es necesario para instalar React, ReactDOM, y otras dependencias (librerías UI, herramientas de build, testing, etc.) y para ejecutar scripts como ***npm start*** o ***npm build***.

Por qué son necesarios en proyectos de React:

React se distribuye como un paquete NPM, por lo que necesitas NPM para instalarlo.

El entorno de desarrollo y el build final dependen de NodeJS para ejecutar herramientas como Vite, Webpack, Babel o CRA.

Sin NodeJS y NPM no podrías usar la mayoría del ecosistema de librerías y automatizaciones que hacen posible trabajar con React de forma eficiente.

4. JSX en React

Reglas de JSX

Regla	Ejemplo
Un elemento padre	... o <>...</> (Fragment)
Atributos camelCase	className, onClick, onSubmit
Cerrar tags	<img />, <input />
JavaScript con {}	{nombre}, {edad > 18 ? "Mayor" : "Menor"}

```
const Perfil = ({ usuario }) => {
    const esAdulto = usuario.edad >= 18;

    return (
        <>
            <div className="perfil">
                <img
                    src={usuario.avatar}
                    alt={`Avatar de ${usuario.nombre}`}
                />
```

```
                <h1>{usuario.nombre}</h1>

                {/* Renderizado condicional */}
                {esAdulto && <span
classNAme="badge">Adulto</span>}

                {/* Lista de elementos */}
                <ul>
                    {usuario.habilidades.map(habilidad => (
                        <li
key={habilidad.id}>{habilidad.nombre}</li>
                    ))}
                </ul>

                {/* Inline styles */}
                <div style={{
                    backgroundColor: esAdulto ? 'green' :
'orange',
                    padding: '10px'
                }}>
                    Estado: {esAdulto ? "Verificado" :
"Pendiente"}
                </div>
            </div>
        </>
    );
};
```

5. Props

Tipos de Props

```
// Props básicos
const Boton = ({ texto, onClick, tipo = "primary", deshabilitado
= false }) => {
```

```
    return (
        <button
            type="button"
            className={`btn btn-${tipo}`}
            onClick={onClick}
            disabled={deshabilitado}
        >
            {texto}
        </button>
    );
};

// Props children
const Modal = ({ children, visible, onClose }) => {
    if (!visible) return null;

    return (
        <div className="modal-overlay" onClick={onClose}>
            <div className="modal-content" onClick={e =>
e.stopPropagation()}>
                <button className="close"
onClick={onClose}>×</button>
                {children}
            </div>
        </div>
    );
};

// Uso
const App = () => {
    return (
        <Modal visible={true} onClose={() =>
console.log("cerrar")}>
            <h2>Título del Modal</h2>
            <p>Contenido del modal...</p>
```

```
            <Boton texto="Aceptar" onClick={() => {}} />
        </Modal>
    );
};
```

6. useState Hook

```
import { useState } from 'react';

const Contador = () => {
    const [count, setCount] = useState(0);

    const incrementar = () => setCount(count + 1);
    const decrementar = () => setCount(count - 1);
    const reset = () => setCount(0);

    return (
        <div className="contador">
            <h2>Contador: {count}</h2>
            <div className="botones">
                <button onClick={decrementar}>-</button>
                <button onClick={reset}>Reset</button>
                <button onClick={incrementar}>+</button>
            </div>
        </div>
    );
};

// Estado con objetos
const FormularioUsuario = () => {
    const [usuario, setUsuario] = useState({
        nombre: '',
        email: '',
        edad: 0
    });
```

```
    const handleChange = (campo, valor) => {
        setUsuario(prevUsuario => ({
            ...prevUsuario,
            [campo]: valor
        }));
    };

    return (
        <form>
            <input
                value={usuario.nombre}
                onChange={e => handleChange('nombre',
e.target.value)}
                placeholder="Nombre"
            />
            <input
                value={usuario.email}
                onChange={e => handleChange('email',
e.target.value)}
                placeholder="Email"
                type="email"
            />
            <input
                value={usuario.edad}
                onChange={e => handleChange('edad',
parseInt(e.target.value))}
                placeholder="Edad"
                type="number"
            />
        </form>
    );
};
```

7. useEffect Hook

```
import { useState, useEffect } from 'react';

// Efecto al montar y desmontar
const ComponenteConEfecto = () => {
    const [datos, setDatos] = useState([]);
    const [cargando, setCargando] = useState(true);

    useEffect(() => {
        // Efecto (ejecuta al montar)
        console.log("Componente montado");

        const cargarDatos = async () => {
            try {
                const response = await fetch('/api/datos');
                const data = await response.json();
                setDatos(data);
            } catch (error) {
                console.error("Error cargando datos:", error);
            } finally {
                setCargando(false);
            }
        };

        cargarDatos();

        // Cleanup (ejecuta al desmontar)
        return () => {
            console.log("Componente desmontado - limpieza");
        };
    }, []); // Array vacío = solo al montar/desmontar

    if (cargando) return <p>Cargando...</p>;

    return (
```

```
        <div>
            <h2>Datos cargados:</h2>
            <ul>
                {datos.map(item => (
                    <li key={item.id}>{item.nombre}</li>
                ))}
            </ul>
        </div>
    );
};

// Efecto con dependencias
const BuscadorUsuarios = () => {
    const [termino, setTermino] = useState('');
    const [resultados, setResultados] = useState([]);

    useEffect(() => {
        if (termino.length > 2) {
            const buscarUsuarios = async () => {
                const response = await
fetch(`/api/usuarios?q=${termino}`);
                const usuarios = await response.json();
                setResultados(usuarios);
            };

            // Debounce
            const timer = setTimeout(buscarUsuarios, 500);
            return () => clearTimeout(timer);
        }
    }, [termino]); // Se ejecuta cuando cambia 'termino'

    return (
        <div>
            <input
                value={termino}
```

```
                onChange={e => setTermino(e.target.value)}
                placeholder="Buscar usuarios..."
            />
            <div>
                {resultados.map(usuario => (
                    <div key={usuario.id}>{usuario.nombre}</div>
                ))}
            </div>
        </div>
```

8. Manejo de Eventos

```
const FormularioCompleto = () => {
    const [formData, setFormData] = useState({
        nombre: '',
        email: '',
        mensaje: ''
    });

    const [errores, setErrores] = useState({});

    const handleInputChange = (e) => {
        const { name, value } = e.target;
        setFormData(prev => ({
            ...prev,
            [name]: value
        }));

        // Limpiar error del campo
        if (errores[name]) {
            setErrores(prev => ({
                ...prev,
                [name]: ''
            }));
        }
```

```
    };

    const validarFormulario = () => {
        const nuevosErrores = {};

        if (!formData.nombre.trim()) {
            nuevosErrores.nombre = "El nombre es requerido";
        }

        if (!formData.email.includes('@')) {
            nuevosErrores.email = "Email inválido";
        }

        if (formData.mensaje.length < 10) {
            nuevosErrores.mensaje = "El mensaje debe tener al
menos 10 caracteres";
        }

        setErrores(nuevosErrores);
        return Object.keys(nuevosErrores).length === 0;
    };

    const handleSubmit = (e) => {
        e.preventDefault();

        if (validarFormulario()) {
            console.log("Formulario válido:", formData);
            // Enviar datos...
        }
    };

    return (
        <form onSubmit={handleSubmit}>
            <div>
                <input
```

```
                    name="nombre"
                    value={formData.nombre}
                    onChange={handleInputChange}
                    placeholder="Nombre"
                />
                {errores.nombre && <span
classNamе="error">{errores.nombre}</span>}
            </div>

            <div>
                <input
                    name="email"
                    type="email"
                    value={formData.email}
                    onChange={handleInputChange}
                    placeholder="Email"
                />
                {errores.email && <span
classNamе="error">{errores.email}</span>}
            </div>

            <div>
                <textarea
                    name="mensaje"
                    value={formData.mensaje}
                    onChange={handleInputChange}
                    placeholder="Mensaje"
                />
                {errores.mensaje && <span
classNamе="error">{errores.mensaje}</span>}
            </div>

            <button type="submit">Enviar</button>
        </form>
    );
```

```
};
```

PREPARACIÓN PARA REACT NATIVE

Diferencias Clave con React Web

Aspecto	React Web	React Native
Elementos	, , <button>	<View>, <Text>, <TouchableOpacity>
Estilos	CSS, clases	StyleSheet, objetos
Navegación	React Router	React Navigation
Eventos	onClick, onChange	onPress, onChangeText
Layouts	CSS Grid, Flexbox	Principalmente Flexbox

Conceptos que se Mantienen

- Componentes funcionales
- Hooks (useState, useEffect, etc.)
- Props y state
- JSX (con adaptaciones)
- Ciclo de vida de componentes
- Manejo de eventos
- Programación funcional

Conceptos Nuevos en React Native

- Componentes nativos específicos
- StyleSheet para estilos
- Navegación entre pantallas
- APIs nativas (cámara, GPS, etc.

- Gestos táctiles
- Optimización para móviles

Recursos Adicionales

Documentación Oficial

- **JavaScript:** MDN Web Docs
- **React:** React Official Docs
- **React Native:** React Native Docs

Herramientas Recomendadas

Herramienta	Propósito	Link
VS Code	Editor de código	código.visualstudio.com
Node.js	Runtime JavaScript	nodejs.org
Git	Control de versiones	git-scm.com
Chrome DevTools	Debugging	Incluido en Chrome

Extensiones VS Code Esenciales

- ES7+ React/Redux/React-Native snippets
- Prettier - Code formatter
- ESLint
- Bracket Pair Colorizer
- Auto Rename Tag



Bibliografía utilizada y sugerida

Mozilla Foundation. JavaScript | MDN Web Docs.
https://developer.mozilla.org/es/docs/Web/JavaScript
Actualizado el 15 de enero de 2025.

Meta Platforms. React – The library for web and native user interfaces.
https://react.dev/
Actualizado el 10 de enero de 2025.

Meta Platforms. React Native · Learn once, write anywhere.
https://reactnative.dev/
Actualizado el 12 de enero de 2025.

Mozilla Foundation. Array - JavaScript | MDN.
https://developer.mozilla.org/es/docs/Web/JavaScript/Reference
Actualizado el 20 de diciembre de 2024.

Mozilla Foundation. Promise - JavaScript | MDN.
https://developer.mozilla.org/es/docs/Web/JavaScript/Reference
Actualizado el 18 de diciembre de 2024.

Meta Platforms. Using the State Hook – React.
https://react.dev/reference/react/useState
Actualizado el 5 de enero de 2025.

Meta Platforms. Using the Effect Hook – React.
https://react.dev/reference/react/useEffect
Actualizado el 5 de enero de 2025.

Mozilla Foundation. Destructuring assignment - JavaScript | MDN.
https://developer.mozilla.org/es/docs/Web/JavaScript/Reference
Actualizado el 22 de diciembre de 2024.
Mozilla Foundation. Arrow function expressions - JavaScript | MDN.
https://developer.mozilla.org/es/docs/Web/JavaScript/Reference
Actualizado el 28 de diciembre de 2024.

Mozilla Foundation. Template literals (Template strings) - JavaScript | MDN.
https://developer.mozilla.org/es/docs/Web/JavaScript/Reference
Actualizado el 15 de diciembre de 2024.

Mozilla Foundation. Spread syntax (...) - JavaScript | MDN.
https://developer.mozilla.org/es/docs/Web/JavaScript/Reference
Actualizado el 10 de enero de 2025.

Mozilla Foundation. async function - JavaScript | MDN.
https://developer.mozilla.org/es/docs/Web/JavaScript/Reference
Actualizado el 25 de diciembre de 2024.

Mozilla Foundation. import - JavaScript | MDN.
https://developer.mozilla.org/es/docs/Web/JavaScript/Reference
Actualizado el 30 de diciembre de 2024.

Mozilla Foundation. export - JavaScript | MDN.
https://developer.mozilla.org/es/docs/Web/JavaScript/Reference
Actualizado el 30 de diciembre de 2024.